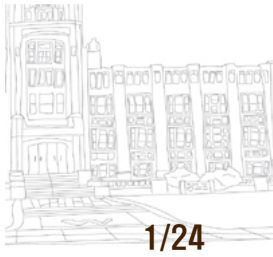
8장. 인터페이스

이것이 자바다



Contents

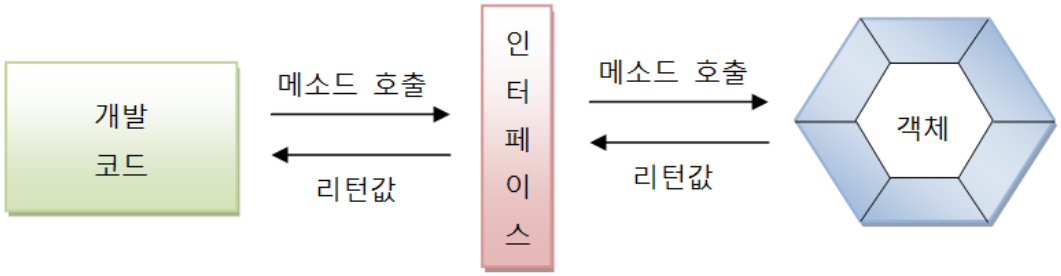
- 1절. 인터페이스의 역할
- 2절. 인터페이스 선언
- 3절. 인터페이스 구현
- 4절. 인터페이스 사용
- 5절. 타입변환과 다형성
- 6절. 인터페이스 상속
- 7절. 디폴트 메소드와 인터페이스 확장



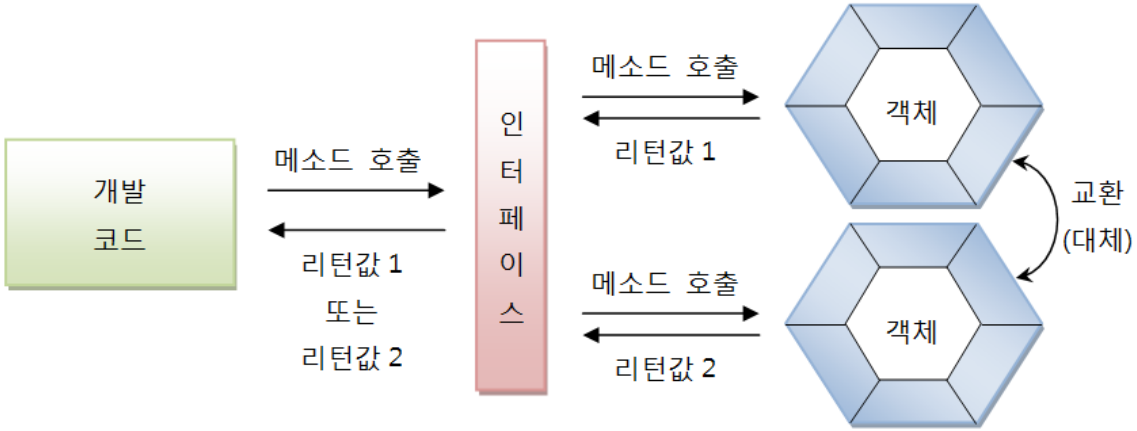
1절. 인터페이스의 역할

❖ 인터페이스란?

- 개발 코드와 객체가 서로 통신하는 접점
  - 개발 코드는 **인터페이스의 메소드만 알고 있으면 OK**



- 인터페이스의 역할
  - 개발 코드가 객체에 종속되지 않게 -> 객체 교체할 수 있도록 하는 역할
  - 개발 코드 변경 없이 리턴값 또는 실행 내용이 다양해 질 수 있음 (다형성)



2절. 인터페이스 선언

❖ 인터페이스 선언

- 인터페이스 이름 – 자바 식별자 작성 규칙에 따라 작성

- 소스 파일 생성
  - 인터페이스 이름과 대소문자가 동일한 소스 파일 생성

- 인터페이스 선언

```
[ public ] interface  인터페이스명 { ... }
```



2절. 인터페이스 선언

❖ 인터페이스 선언

- 인터페이스의 구성 멤버 (p.346)

```
interface 인터페이스명 {
  //상수
  타입 상수명 = 값;
  //추상 메소드
  타입 메소드명(매개변수,...);
  //디폴트 메소드
  default 타입 메소드명(매개변수,...) {...}
  //정적 메소드
  static 타입 메소드명(매개변수) {...}
}
```



2절. 인터페이스 선언

❖ 상수 필드 선언

- 인터페이스는 상수 필드만 선언 가능
  - 데이터 저장하지 않음

- 인터페이스에 선언된 필드는 모두 public static final
  - 자동적으로 컴파일 과정에서 붙음

- 상수명은 대문자로 작성
  - 서로 다른 단어로 구성되어 있을 경우에는 언더 바(_)로 연결

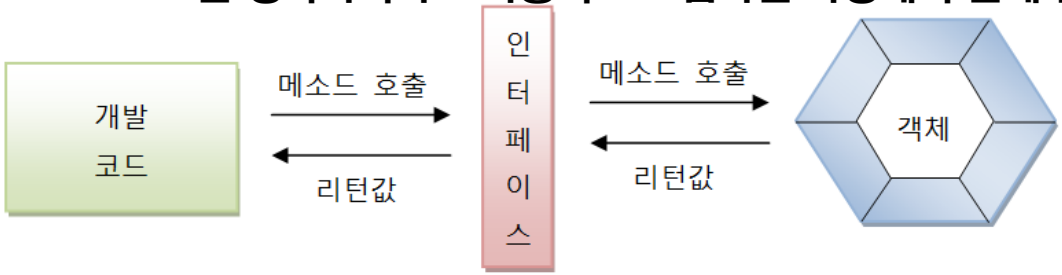
- 선언과 동시에 초기값 지정
  - static { } 블록 작성 불가 - static {} 으로 초기화 불가

```
1 public interface RemoteControl {
2     int MAX_VOLUME = 10;
3     int MIN_VOLUME = 0;
4 }
```

2절. 인터페이스 선언

❖ 추상 메소드 선언

- 인터페이스 통해 호출된 메소드는 최종적으로 객체에서 실행
  - 인터페이스의 메소드는 기본적으로 실행 블록이 없는 추상 메소드로 선언
  - public abstract를 생략하더라도 자동적으로 컴파일 과정에서 붙게 됨



추상메소드
(메소드 선언부)
(호출 방법만 기술)

재정의된 메소드
(실제 실행 메소드)

[public abstract] 리턴타입 메소드명(매개변수, ...);

```
3  public interface RemoteControl {
4      //상수
5      int MAX_VOLUME = 10;
6      int MIN_VOLUME = 0;
7
8      //추상 메소드
9      void turnOn();
10     void turnOff();
11     void setVolume(int volume);
12 }
13
```

메소드 선언부만 작성

2절. 인터페이스 선언

❖ 디폴트 메소드 선언

- 자바8에서 추가된 인터페이스의 새로운 멤버

```
[public] default 리턴타입 메소드명(매개변수, ...) { ... }
```

- 실행 블록을 가지고 있는 메소드
- default 키워드를 반드시 붙여야
- 기본적으로 public 접근 제한
  - 생략하더라도 컴파일 과정에서 자동 붙음

```
public interface RemoteControl {
    //상수
    int MAX_VOLUME = 10;
    int MIN_VOLUME = 0;

    //추상 메소드
    void turnOn();
    void turnOff();
    void setVolume(int volume);

    //디폴트 메소드
    default void setMute(boolean mute) {
        if(mute) {
            System.out.println("무음 처리합니다.");
        } else {
            System.out.println("무음 해제합니다.");
        }
    }
}
```

2절. 인터페이스 선언

❖ 정적 메소드 선언

- 자바8에서 추가된 인터페이스의 새로운 멤버

[public] static 리턴타입 메소드명(매개변수, ...) { ... }

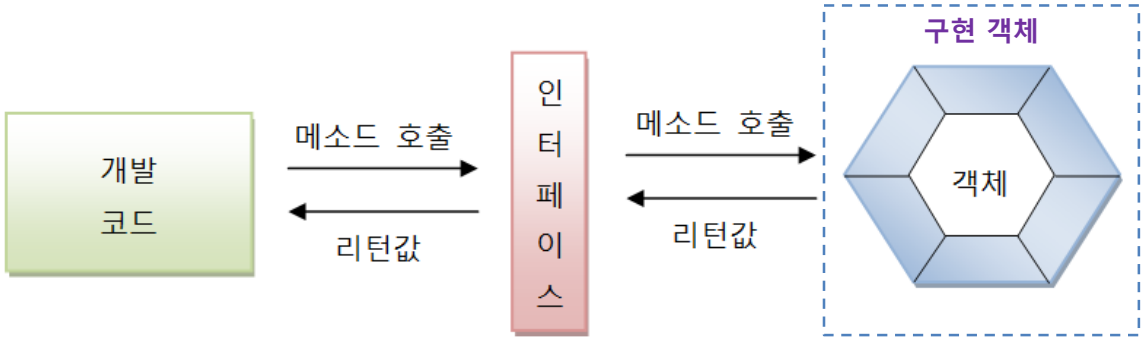
```
public interface RemoteControl {
    static void changeBattery() {
        System.out.println("건전지를 교환합니다.");
    }
}
```

```
6       //추상 메소드
7       void turnOn();
8       void turnOff();
9       void setVolume(int volume);
10
11      //디폴트 메소드
12      default void setMute(boolean mute) {
13          if(mute) {
14              System.out.println("무음 처리합니다.");
15          } else {
16              System.out.println("무음 해제합니다.");
17          }
18      }
19
20      //정적 메소드
21      static void changeBattery() {
22          System.out.println("건전지를 교환합니다.");
23      }
24  }
25
```

3절. 인터페이스 구현

❖ 구현 객체와 구현 클래스

- 인터페이스의 추상 메소드 대한 실체 메소드를 가진 객체 = 구현 객체



- 구현 객체를 생성하는 클래스 = 구현 클래스

3절. 인터페이스 구현

❖ 구현 클래스 선언

- 자신의 객체가 인터페이스 타입으로 사용할 수 있음
  - implements 키워드로 명시

```
public class 구현클래스명 implements 인터페이스명 {
    //인터페이스에 선언된 추상 메소드의 실체 메소드 선언
}
```

❖ 추상 메소드의 실체 메소드를 작성하는 방법

- 메소드의 선언부가 정확히 일치해야
- 인터페이스의 모든 추상 메소드를 재정의하는 실체 메소드 작성해야
  - 일부만 재정의할 경우, 추상 클래스로 선언 + abstract 키워드 붙임



Remotecontrol.java_메소드 선언

```java
public interface RemoteControl {
    //상수
    int MAX_VOLUME = 10;
    int MIN_VOLUME = 0;

    //추상 메소드
    void turnOn();
    void turnOff();
    void setVolume(int volume);

    //디폴트 메소드
    default void setMute(boolean mute) {
        if(mute) {
            System.out.println("무음 처리합니다.");
        } else {
            System.out.println("무음 해제합니다.");
        }
    }

    //정적 메소드
    static void changeBattery() {
        System.out.println("건전지를 교환합니다.");
    }
}
```



Television.java_구현 클래스

```java
public class Television implements RemoteControl {
    //필드
    private int volume;

    //turnOn() 추상 메소드의 실체 메소드
    public void turnOn() {
        System.out.println("TV를 켭니다.");
    }
    //turnOff() 추상 메소드의 실체 메소드
    public void turnOff() {
        System.out.println("TV를 끕니다.");
    }
    //setVolume() 추상 메소드의 실체 메소드
    public void setVolume(int volume) {
        if(volume>RemoteControl.MAX_VOLUME) {
            this.volume = RemoteControl.MAX_VOLUME;
        } else if(volume<RemoteControl.MIN_VOLUME) {
            this.volume = RemoteControl.MIN_VOLUME;
        } else {
            this.volume = volume;
        }
        System.out.println("현재 TV 볼륨: " + volume);
    }
}

```



Audio.java_구현 클래스

```java
public class Audio implements RemoteControl {
    //필드
    private int volume;

    //turnOn() 추상 메소드의 실체 메소드
    public void turnOn() {
        System.out.println("Audio를 켭니다.");
    }
    //turnOff() 추상 메소드의 실체 메소드
    public void turnOff() {
        System.out.println("Audio를 끕니다.");
    }
    //setVolume() 추상 메소드의 실체 메소드
    public void setVolume(int volume) {
        if(volume>RemoteControl.MAX_VOLUME) {
            this.volume = RemoteControl.MAX_VOLUME;
        } else if(volume<RemoteControl.MIN_VOLUME) {
            this.volume = RemoteControl.MIN_VOLUME;
        } else {
            this.volume = volume;
        }
        System.out.println("현재 Audio 볼륨: " + volume);
    }
}

```



Remotecontroller.java_인터페이스 변수에 구현 객체 대입

```
1  public class RemoteControlExample {
2      public static void main(String[] args) {
3          RemoteControl rc;
4          rc = new Television();
5          rc = new Audio();
6
7          RemoteControl.changeBattery();
8      }
9  }
10
```

```
C:₩Windows₩system32₩cmd.exe
건전지를 교환합니다.
계속하려면 아무 키나 누르십시오 . . .
```



3절. 인터페이스 구현

❖ 익명 구현 객체

- 명시적인 구현 클래스 작성 생략하고 바로 구현 객체를 얻는 방법
  - 이름 없는 구현 클래스 선언과 동시에 객체 생성

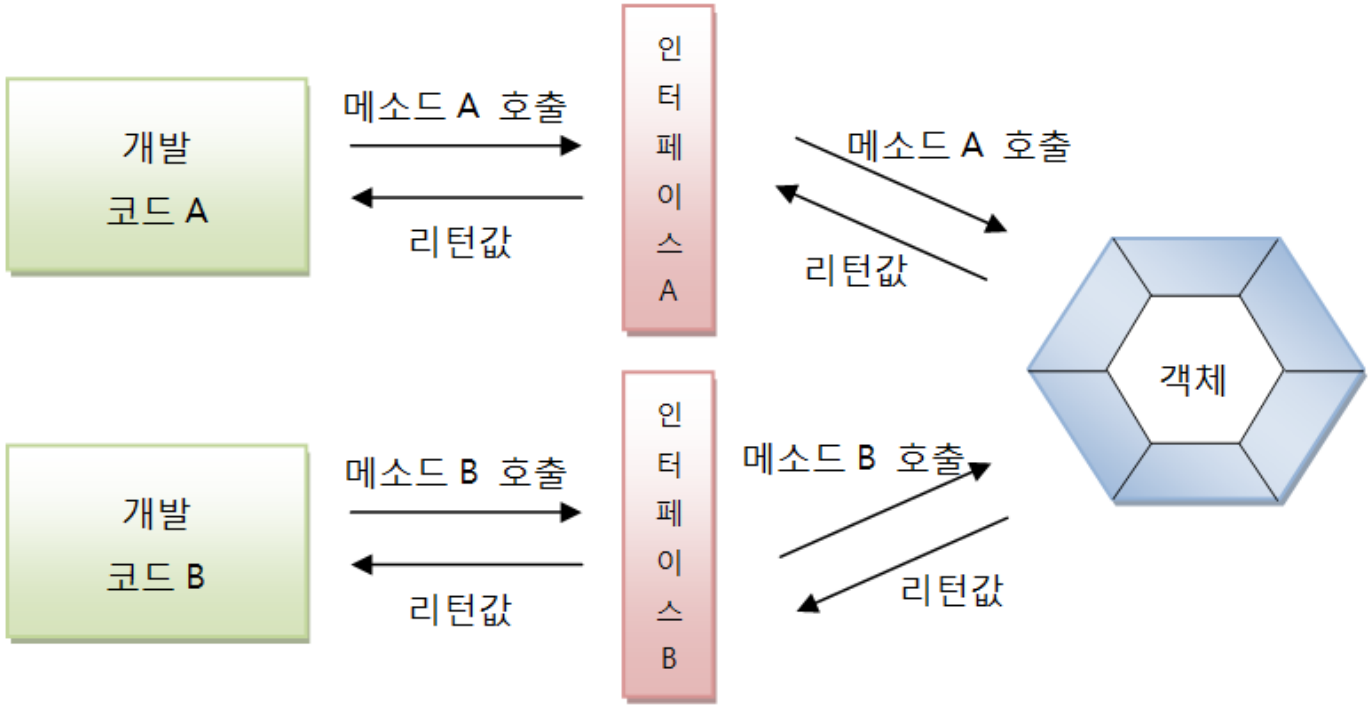
```
인터페이스 변수 = new 인터페이스() {
  //인터페이스에 선언된 추상 메소드의 실체 메소드 선언
};
```

  - 인터페이스의 추상 메소드들을 모두 재정의하는 실체 메소드가 있어야
  - 추가적으로 필드와 메소드 선언 가능하나 익명 객체 안에서만 사용
  - 인터페이스 변수로 접근 불가

```
1 public class RemoteControlExample {
2     public static void main(String[] args) {
3         RemoteControl rc = new RemoteControl() {
4             public void turnOn() { /*실행문*/ }
5             public void turnOff() { /*실행문*/ }
6             public void setVolume(int volume) { /*실행문*/ }
7         };
8     }
9 }
10
```

3절. 인터페이스 구현

❖ 다중 인터페이스 구현 클래스




```
public class 구현클래스명 implements 인터페이스 A, 인터페이스 B {
    //인터페이스 A 에 선언된 추상 메소드의 실체 메소드 선언
    //인터페이스 B 에 선언된 추상 메소드의 실체 메소드 선언
}
```

Searchable.java_인터페이스

```java
public interface Searchable {
    void search(String url);
}
```

```java
public class SmartTelevision implements RemoteControl, Searchable {
    private int volume;

    public void turnOn() {
        System.out.println("TV를 켭니다.");
    }
    public void turnOff() {
        System.out.println("TV를 끕니다.");
    }
    public void setVolume(int volume) {
        if(volume>RemoteControl.MAX_VOLUME) {
            this.volume = RemoteControl.MAX_VOLUME;
        } else if(volume<RemoteControl.MIN_VOLUME) {
            this.volume = RemoteControl.MIN_VOLUME;
        } else {
            this.volume = volume;
        }
        System.out.println("현재 TV 볼륨: " + volume);
    }

    public void search(String url) {
        System.out.println(url + "을 검색합니다.");
    }
}
```

SmartTelevision.java_다중 인터페이스 구현 클래스

4절. 인터페이스 사용

❖ 인터페이스에 구현 객체를 대입하는 방법 (p.357~358)

```
인터페이스 변수;
변수 = 구현객체;
```

```
인터페이스 변수 = 구현객체;
```

```
RemoteControl rc;
rc = new Television();
rc = new Audio();
```



4절. 인터페이스 사용

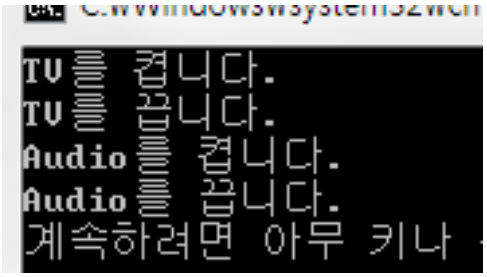
❖ 추상 메소드 사용

```
RemoteControl rc = new Television();
rc.turnOn();    → Television 의 turnOn() 실행
rc.turnOff();   → Television 의 turnOff() 실행
```



RemotecontrolExample.java_인터페이스 사용

```
1  public class RemoteControlExample {
2      public static void main(String[] args) {
3          RemoteControl rc = null;
4
5          rc = new Television();
6          rc.turnOn();
7          rc.turnOff();
8
9          rc = new Audio();
10         rc.turnOn();
11         rc.turnOff();
12     }
13 }
14
```



관련 클래스

- Audio.java
- RemoteControl.java
- RemoteControlExample.java
- Television.java

4절. 인터페이스 사용

❖ 디폴트 메소드 사용

- 인터페이스만으로는 사용 불가
  - 구현 객체가 인터페이스에 대입되어야 호출할 수 있는 인스턴스 메소드

- 모든 구현 객체가 가지고 있는 기본 메소드로 사용
  - 필요에 따라 구현 클래스가 디폴트 메소드 재정의해 사용



Audio.java_구현 클래스

```java
public class Audio implements RemoteControl {
    //필드
    private int volume;
    private boolean mute;

    //turnOn() 추상 메소드의 실체 메소드
    public void turnOn() {
        System.out.println("Audio를 켭니다.");
    }
    //turnOff() 추상 메소드의 실체 메소드
    public void turnOff() {
        System.out.println("Audio를 끕니다.");
    }
    //setVolume() 추상 메소드의 실체 메소드
    public void setVolume(int volume) {
        if(volume>RemoteControl.MAX_VOLUME) {
            this.volume = RemoteControl.MAX_VOLUME;
        } else if(volume<RemoteControl.MIN_VOLUME) {
            this.volume = RemoteControl.MIN_VOLUME;
        } else {
            this.volume = volume;
        }
        System.out.println("현재 Audio 볼륨: " + volume);
    }

    @Override
    public void setMute(boolean mute) {
        this.mute = mute;
        if(mute) {
            System.out.println("Audio 무음 처리합니다.");
        } else {
            System.out.println("Audio 무음 해제합니다.");
        }
    }
}
```



RemotecontrolExample.java_디폴트 메소드 사용

```
1  public class RemoteControlExample {
2      public static void main(String[] args) {
3          RemoteControl rc = null;
4
5          rc = new Television();
6          rc.turnOn();
7          rc.setMute(true);
8
9          rc = new Audio();
10         rc.turnOn();
11         rc.setMute(true);
12     }
13 }
```

```
TV를 켭니다.
무음 처리합니다.
Audio를 켭니다.
Audio 무음 처리합니다.
계속하려면 아무 키나 누르십시오 . .
```

관련 클래스

- Audio.java
- RemoteControl.java
- RemoteControlExample.java
- Television.java

4절. 인터페이스 사용

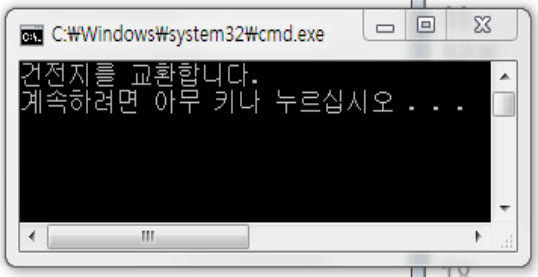
❖ 정적 메소드 사용

- 인터페이스로 바로 호출 가능

```
1 public class RemoteControlExample {
2     public static void main(String[] args) {
3         RemoteControl.changeBattery();
4     }
5 }
6
```

```
1 public interface RemoteControl {
2     //상수
3     int MAX_VOLUME = 10;
4     int MIN_VOLUME = 0;
5
6     //추상 메소드
7     void turnOn();
8     void turnOff();
9     void setVolume(int volume);
10
    //디폴트 메소드
    default void setMute(boolean mute) {
        if(mute) {
            System.out.println("무음 처리합니다.");
        } else {
            System.out.println("무음 해제합니다.");
        }
    }

    //정적 메소드
    static void changeBattery() {
        System.out.println("건전지를 교환합니다.");
    }
}
```



5절. 타입변환과 다형성

❖ 다형성 (p.362~364)

- 하나의 타입에 여러 가지 객체 대입해 다양한 실행 결과를 얻는 것

- 다형성을 구현하는 기술
  - 상속 또는 인터페이스의 자동 타입 변환(Promotion)
  - 오버라이딩(Overriding)

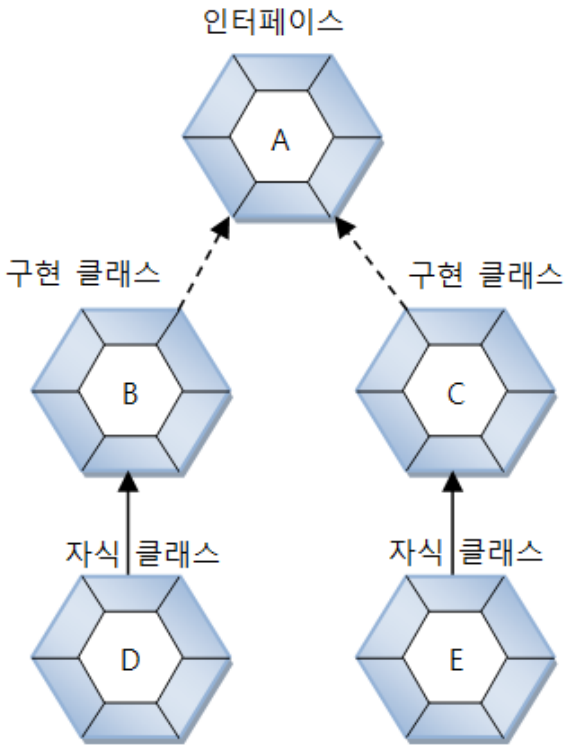
- 다형성의 효과
  - 다양한 실행 결과를 얻을 수 있음
  - 객체를 부품화시킬 수 있어 유지보수 용이 (메소드의 매개변수로 사용)

5절. 타입변환과 다형성

❖ 자동 타입 변환(Promotion)

자동 타입 변환

인터페이스 변수 = 구현객체;




```
B b = new B();
C c = new C();
D d = new D();
E e = new E();
```

```
A a1 = b;  (가능)
A a2 = c;  (가능)
A a3 = d;  (가능)
A a4 = e;  (가능)
```

5절. 타입변환과 다형성

❖ 필드의 다형성 (p.365~368)

[다형성은 객체를 부품화시킨다]




```
public interface Tire {
    public void roll();
}
```

```
public class HankookTire implements Tire {
    @Override
    public void roll() {
        System.out.println("한국 타이어가 굴러갑니다.");
    }
}
```

```
public class Car {
    Tire frontLeftTire = new HankookTire();
    Tire frontRightTire = new HankookTire();
    Tire backLeftTire = new HankookTire();
    Tire backRightTire = new HankookTire();

    void run() {
        frontLeftTire.roll();
        frontRightTire.roll();
        backLeftTire.roll();
        backRightTire.roll();
    }
```

```
Car myCar = new Car();
myCar.frontLeftTire = new KumhoTire();
myCar.frontRightTire = new KumhoTire();
```

```
myCar.run();
```

Tire.java_인터페이스

```java
public class Car {
    Tire frontLeftTire = new HankookTire();
    Tire frontRightTire = new HankookTire();
    Tire backLeftTire = new HankookTire();
    Tire backRightTire = new HankookTire();

    void run() {
        frontLeftTire.roll();
        frontRightTire.roll();
        backLeftTire.roll();
        backRightTire.roll();
    }
}
```

```java
public class HankookTire implements Tire {
    @Override
    public void roll() {
        System.out.println("한국 타이어가 굴러갑니다.");
    }
}
```

HankookTire.java_구현 클래스

```java
public class KumhoTire implements Tire {
    @Override
    public void roll() {
        System.out.println("금호 타이어가 굴러갑니다.");
    }
}
```

KumhoTire.java_구현 클래스

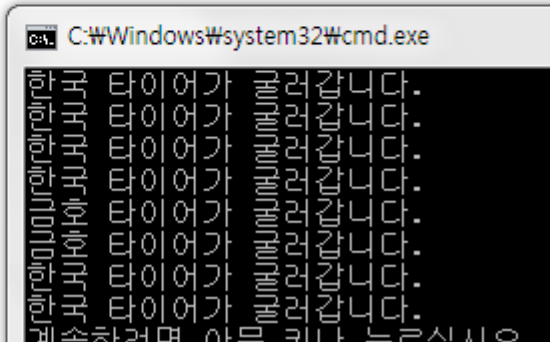
Car.java_필드 다형성

```
1  public class Car {
2      Tire frontLeftTire = new HankookTire();
3      Tire frontRightTire = new HankookTire();
4      Tire backLeftTire = new HankookTire();
5      Tire backRightTire = new HankookTire();
6
7      void run() {
8          frontLeftTire.roll();
9          frontRightTire.roll();
10         backLeftTire.roll();
11         backRightTire.roll();
12     }
13 }
14
```



CarExample.java_필드 다형성 테스트

```
1  public class CarExample {
2      public static void main(String[] args) {
3          Car myCar = new Car();
4
5          myCar.run();
6          myCar.frontLeftTire = new KumhoTire();
7          myCar.frontRightTire = new KumhoTire();
8
9          myCar.run();
10     }
11 }
12
```



관련 클래스

- Car.java
- CarExample.java
- HankookTire.java
- KumhoTire.java
- Tire.java

5절. 타입변환과 다형성

❖ 인터페이스 배열로 구현한 객체 관리

```
Tire[] tires = {
    new HankookTire(),
    new HankookTire(),
    new HankookTire(),
    new HankookTire()
};
```

```
tires[1] = new KumhoTire();
```

```
void run() {
    for(Tire tire : tires) {
        tire.roll();
    }
}
```

```
1  public class Car {
2      Tire frontLeftTire = new HankookTire();
3      Tire frontRightTire = new HankookTire();
4      Tire backLeftTire = new HankookTire();
5      Tire backRightTire = new HankookTire();
6
7      void run() {
8          frontLeftTire.roll();
9          frontRightTire.roll();
10         backLeftTire.roll();
11         backRightTire.roll();
12     }
13 }
```

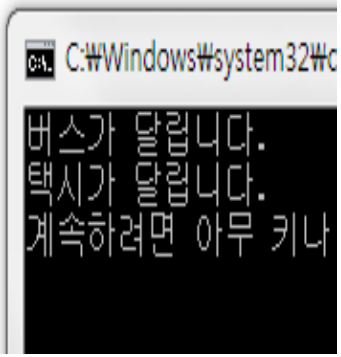


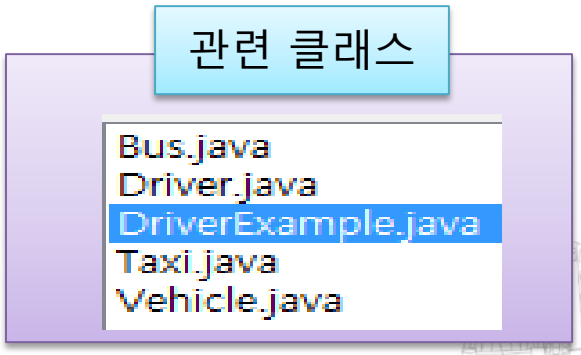
```
void run() {
    for(Tire tire : tires) {
        tire.roll();
    }
}
```

5절. 타입변환과 다형성

- **매개변수의 다형성**
  - 매개 변수의 타입이 인터페이스인 경우
    - 어떠한 구현 객체도 매개값으로 사용 가능
    - 구현 객체에 따라 메소드 실행결과 달라짐

```
1 public class DriverExample {
2     public static void main(String[] args) {
3         Driver driver = new Driver();
4
5         Bus bus = new Bus();
6         Taxi taxi = new Taxi();
7
8         driver.drive(bus);
9         driver.drive(taxi);
10    }
11 }
12
```



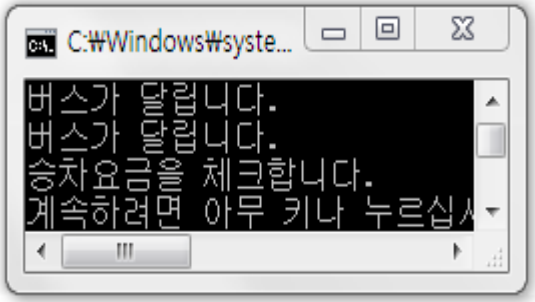


5절. 타입변환과 다형성

❖ 강제 타입 변환(Casting)

- 인터페이스 타입으로 자동 타입 변환 후, 구현 클래스 타입으로 변환
  - 필요성: 구현 클래스 타입에 선언된 다른 멤버 사용하기 위해

```
1  public class VehicleExample {
2      public static void main(String[] args) {
3          Vehicle vehicle = new Bus();
4
5          vehicle.run();
6          //vehicle.checkFare(); (x)
7
8          Bus bus = (Bus) vehicle;  //강제타입변환
9
10         bus.run();
11         bus.checkFare();
12     }
13 }
14
```



관련 클래스

Bus.java
Vehicle.java
VehicleExample.java

5절. 타입변환과 다형성

❖ 객체 타입 확인(instanceof 연산자) (p.375~377)

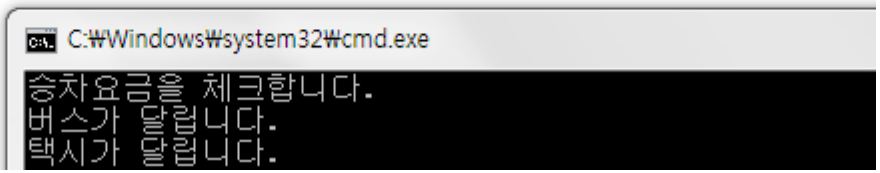
- 강제 타입 변환 전 구현 클래스 타입 조사

```
2  public class Driver {
3      public void drive(Vehicle vehicle) {
4          if(vehicle instanceof Bus) {
5              Bus bus = (Bus) vehicle;
6              bus.checkFare();
7          }
8          vehicle.run();
9      }
10 }
```

Driver.java_객체 타입 확인

객체 타입 확인(Instanceof)

```
1  public class DriverExample {
2      public static void main(String[] args) {
3          Driver driver = new Driver();
4
5          Bus bus = new Bus();
6          Taxi taxi = new Taxi();
7
8          driver.drive(bus);
9          driver.drive(taxi);
10     }
11 }
12
```



관련 클래스

- Bus
- Driver
- DriverExample
- Taxi
- Vehicle

6절. 인터페이스 상속

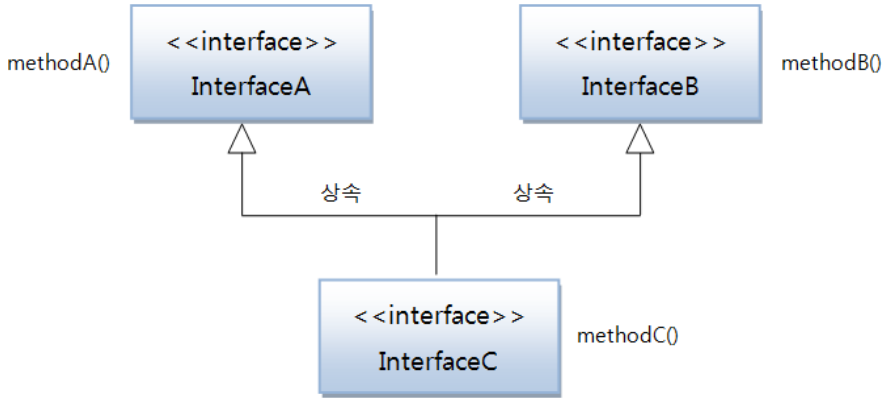
❖ 인터페이스간 상속 가능

```
public interface 하위인터페이스 extends 상위인터페이스1, 상위인터페이스2 { ... }
```

- 하위 인터페이스 구현 클래스는 아래 추상 메소드를 모두 재정의해야
  - 하위 인터페이스의 추상 메소드
  - 상위 인터페이스1의 추상 메소드
  - 상위 인터페이스2의 추상 메소드

```
하위인터페이스 변수 = new 구현클래스(...);
상위인터페이스1 변수 = new 구현클래스(...);
상위인터페이스2 변수 = new 구현클래스(...);
```

- 인터페이스 자동 타입 변환
  - 해당 타입의 인터페이스에 선언된 메소드만 호출 가능



InterfaceA.java_ 부모 인터페이스

```
1 public interface InterfaceA {
2       public void methodA();
3  }
```

InterfaceB.java_ 부모 인터페이스

```
1 public interface InterfaceB {
2       public void methodB();
3  }
```

InterfaceC.java_ 하위 인터페이스

```
1 public interface InterfaceC extends InterfaceA, InterfaceB {
2       public void methodC();
3  }
4
```



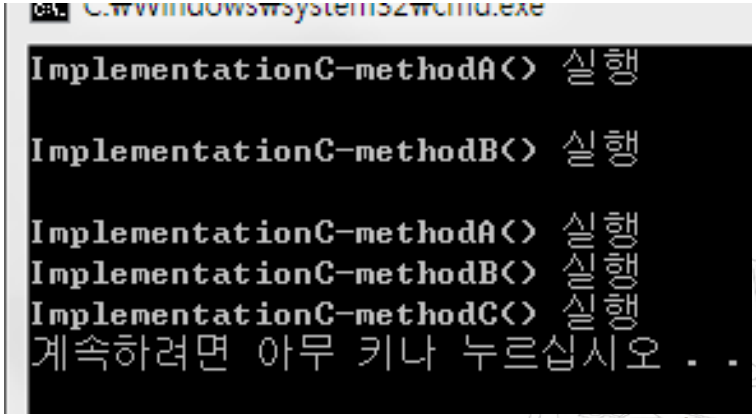
ImplementationC.java_ 하위 인터페이스 구현

```
1  public class ImplementationC implements InterfaceC {
2      public void methodA() {
3          System.out.println("ImplementationC-methodA() 실행");
4      }
5
6      public void methodB() {
7          System.out.println("ImplementationC-methodB() 실행");
8      }
9
10     public void methodC() {
11         System.out.println("ImplementationC-methodC() 실행");
12     }
13 }
```



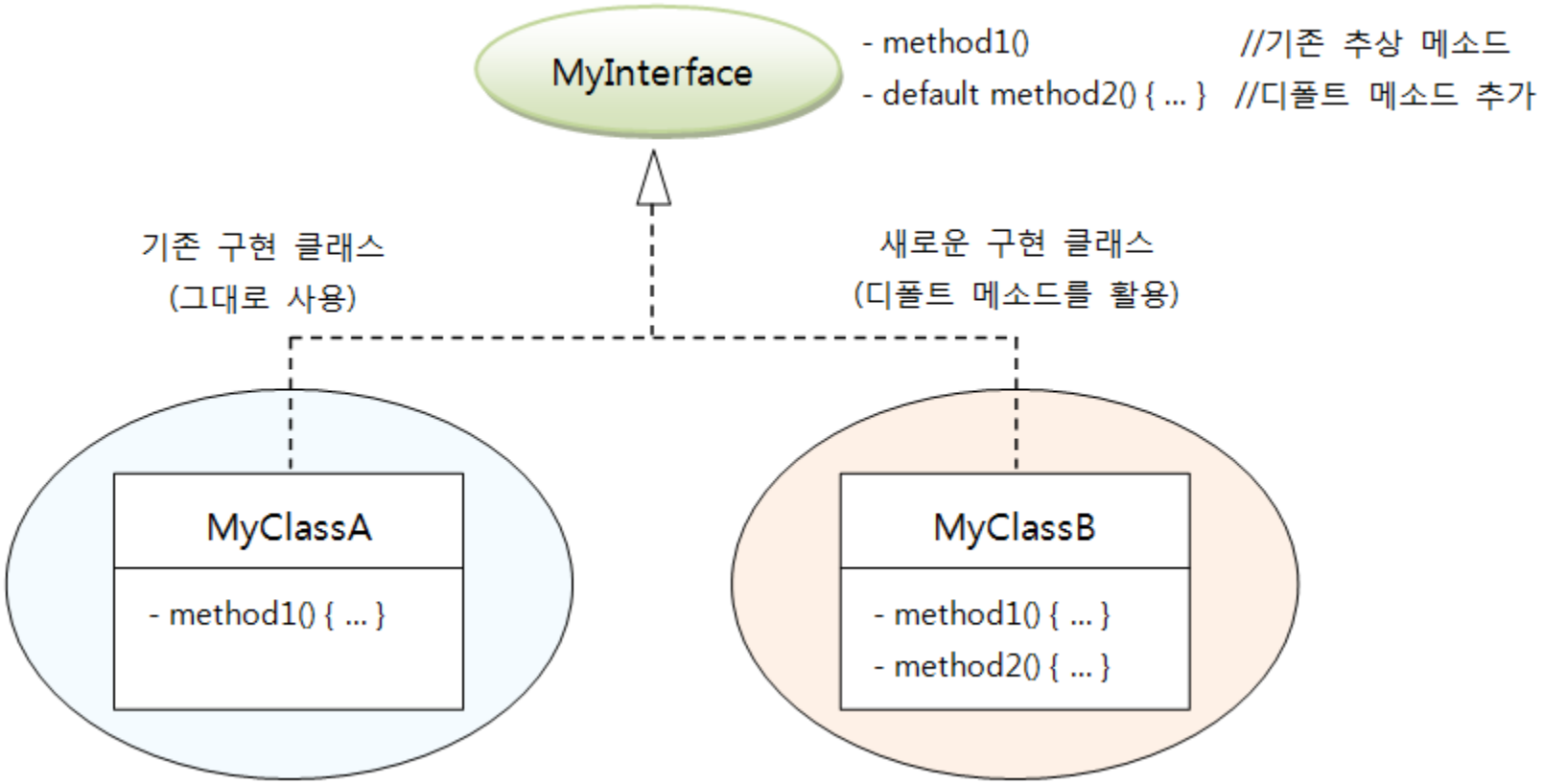
Example.java_ 호출 가능 메소드

```
1  public class Example {
2      public static void main(String[] args) {
3          ImplementationC impl = new ImplementationC();
4
5          InterfaceA ia = impl;
6          ia.methodA();
7          System.out.println();
8
9          InterfaceB ib = impl;
10         ib.methodB();
11         System.out.println();
12
13         InterfaceC ic = impl;
14         ic.methodA();
15         ic.methodB();
16         ic.methodC();
17     }
18 }
```



7절. 디폴트 메소드와 인터페이스 확장

❖ 디폴트 메소드와 확장 메소드 사용하기 (p.379~382)



Myinterface.java_기존 인터페이스-수정 인터페이스

```java
public interface MyInterface {
    public void method1();

    public default void method2() {
        System.out.println("MyInterface-method2 실행");
    }
}
```

MyclassA.java_기존 구현 클래스

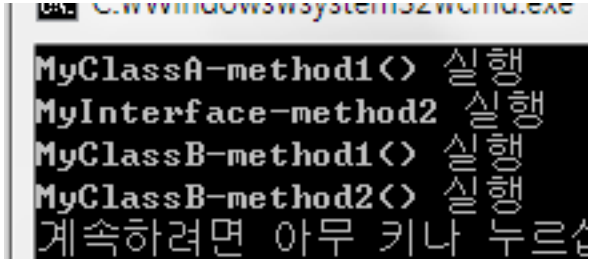
```java
public class MyClassA implements MyInterface {
    @Override
    public void method1() {
        System.out.println("MyClassA-method1() 실행");
    }
}
```

MyclassB.java_새로운 구현 클래스

```java
    @Override
    public void method2() {
        System.out.println("MyClassB-method2() 실행");
    }
}
```

DefaultMethodExample.java_디폴트 메소드 사용

```
1  public class DefaultMethodExample {
2      public static void main(String[] args) {
3          MyInterface mi1 = new MyClassA();
4          mi1.method1();
5          mi1.method2();
6
7          MyInterface mi2 = new MyClassB();
8          mi2.method1();
9          mi2.method2();
10     }
11 }
12
```



관련 클래스

- DefaultMethodExample.java
- MyClassA.java
- MyClassB.java
- MyInterface.java

7절. 디폴트 메소드와 인터페이스 확장

❖ 디폴트 메소드가 있는 인터페이스 상속

- 부모 인터페이스의 디폴트 메소드를 자식 인터페이스에서 활용 방법
  - 디폴트 메소드를 단순히 상속만 받음
  - 디폴트 메소드를 재정의(Override)해서 실행 내용을 변경
  - 디폴트 메소드를 추상 메소드로 재선언

